

15 Asset Class Wise Quantitative Finance Projects A Practical Guide for Aspiring Quants

Amit Kumar Jha , Quant Analyst at UBS

November 20, 2025

```

src/
  python/
    lsm_engine.py
    hw_model.py
    main.py
  cpp/
    include/
      hull_white_model.h
      bermudan_swaption.h
      lsm_regression.h
    src/
      hull_white_model.cpp
      bermudan_swaption.cpp
      main.cpp
    tests/
      test_lsm.cpp
      regression_tests.cpp
notebooks/
  convergence_study.ipynb
  exercise_boundary.ipynb
data/
  model_parameters.json
  market_swaption_quotes.csv

```

Interview Questions with Answers

1. Explain LongstaffSchwartz for Bermudans.

Answer: LongstaffSchwartz is a regression-based Monte Carlo method for optimal stopping problems. It works by:

1. **Forward simulation:** Generate thousands of paths of underlying risk factors (e.g., short rates) using a calibrated model.
2. **Backward induction:** Starting at the final exercise date, calculate the immediate exercise payoff for each path. Then move one step backward in time.
3. **Regression estimation:** At each exercise date, regress the discounted *continuation value* (future payoffs if not exercised now) onto basis functions of current state variables (e.g., swap rate, rate level). This estimates the expected value of waiting.
4. **Optimal decision:** For each path, compare immediate exercise value vs. estimated continuation. Exercise if immediate > continuation.
5. **Pricing:** Discount all optimally exercised payoffs back to time zero and average.

Key insight: It approximates the continuation function via regression, avoiding grid-based methods that suffer from dimensionality. The method is biased low (underestimates true value) because regression error typically reduces estimated continuation value, but converges to true price as paths and basis functions increase.

Resume Bullet

Built dividend-aware Black-Scholes engine supporting continuous yield and discrete cash dividends, computing stable Greeks via automatic differentiation used to price 10k+ single-stock options daily.

```

45         for j in range(i + 1):
46             # Continuation value: hold and receive coupon + expected
               PV
47             continuation = (values[i + 1, j] + values[i + 1, j + 1])
               / 2 / \
48                             (1 + r_tree[i, j] * dt) + self.coupon
49
50             # Check if callable at this time
51             time_to_now = i * dt
52             call_price = self._get_call_price(time_to_now)
53
54             # Issuer will call if continuation > call_price
55             values[i, j] = min(continuation, call_price)
56
57     return values[0, 0]
58
59 def compute_oas(self, market_price, r_tree, n_steps):
60     """Solve for OAS that matches market price"""
61     def price_error(oas):
62         # Shift entire lattice by OAS
63         r_tree_shifted = r_tree + oas / 10000 # OAS in bps
64         model_price = self.price_on_lattice(r_tree_shifted, n_steps)
65         return model_price - market_price
66
67     # Brent method to find OAS
68     try:
69         from scipy.optimize import brentq
70         oas = brentq(price_error, -200, 200, xtol=1e-4) # search
               +/-200bp
71     except:
72         oas = np.nan
73
74     return oas
75
76 def effective_duration(self, r_tree, n_steps, dy=0.0001):
77     """Effective duration with OAS constant"""
78     price = self.price_on_lattice(r_tree, n_steps)
79
80     # Shift rates up
81     r_up = r_tree + dy
82     price_up = self.price_on_lattice(r_up, n_steps)
83
84     # Shift rates down
85     r_down = r_tree - dy
86     price_down = self.price_on_lattice(r_down, n_steps)
87
88     return (price_down - price_up) / (2 * price * dy)
89

```